# Lab 5: konto i synchronizacja (Supabase)

Zaawansowane programowanie na platformę iOS

Na tych zajęciach aplikacja dostaje konto użytkownika. Pierwszy ekran to logowanie. Zapisane filmy trafiają na konto w Supabase, więc są na każdym urządzeniu, na którym użytkownik się zaloguje. Lista dalej działa bez internetu, a zmiany czekają na telefonie do następnej synchronizacji.

Supabase to gotowy backend: baza PostgreSQL, konta (Auth), API REST do każdej tabeli i Row Level Security, czyli zasady, kto widzi który wiersz. Serwera nie piszemy.

Cały kod piszą Państwo sami. Ta instrukcja mówi, co ma powstać i gdzie. Nazwy plików, typów i funkcji proszę zachować, bo według nich sprawdzam prace.

## Zanim Państwo zaczną

1. Lab 4 musi być skończony.
2. Konto na supabase.com i nowy projekt. Darmowy plan wystarczy.
3. Supabase CLI: `brew install supabase/tap/supabase`.
4. W projekcie jest pakiet `supabase-swift` (Project navigator, Package Dependencies) i folder `supabase/`. Jeśli Państwa projekt ich nie ma, bo był pobrany wcześniej:
   - pakiet: File, Add Package Dependencies, adres `https://github.com/supabase/supabase-swift`, reguła Up to Next Major Version od `2.55.3`, produkt `Supabase` dodany do targetu `MyNextMovie`,
   - folder: proszę pobrać aktualny ZIP repozytorium przedmiotu i skopiować z niego cały folder `supabase/` obok `MyNextMovie.xcodeproj`.

## Pliki

Nowe:

| | |
|---|---|
| `MyNextMovie/Services/Account.swift` | typy AccountService i SavedMoviesCloud |
| `MyNextMovie/Services/Supabase.swift` | zapytania do Supabase |
| `MyNextMovie/Storage/SavedMoviesSync.swift` | synchronizacja zapisanych filmów |
| `MyNextMovie/ViewModels/AccountViewModel.swift` | stan konta i synchronizacji |
| `MyNextMovie/Views/SignInView.swift` | ekran logowania |
| `MyNextMovie/Views/AccountView.swift` | zakładka Account |
| `MyNextMovieTests/AccountViewModelTests.swift` | testy konta |
| `MyNextMovieTests/SavedMoviesSyncTests.swift` | testy synchronizacji |

Do zmiany:

| | |
|---|---|
| `MyNextMovie/Storage/StoredModels.swift` | model RemovedSavedMovie, isInCloud w init |
| `MyNextMovie/Storage/Storage.swift` | znacznik usunięcia, czyszczenie po wylogowaniu |
| `MyNextMovie/MyNextMovieApp.swift` | logowanie przed zakładkami, moment synchronizacji |
| `MyNextMovie/Views/Components/Layout.swift` | nazwy nowych ikon |
| `MyNextMovieTests/StorageTests.swift` | testy znacznika i czyszczenia |
| `Config/Secrets.xcconfig` | adres i klucz Supabase |

## Jak działa Supabase

Zapisane filmy są w tabeli `saved_movies`. Jeden wiersz to jeden film zapisany przez jednego użytkownika:

- `user_id` wskazuje konto w `auth.users` i domyślnie wypełnia się id zalogowanego użytkownika. Aplikacja go nie wysyła.
- Klucz główny to para `user_id` i `id`, więc użytkownik ma dany film najwyżej raz.
- Pozostałe kolumny to pola `Movie` w snake_case (`release_date`, `poster_path`) i `saved_at`. Film nie ma osobnej tabeli. Wiersz to kopia danych z TMDB, żeby lista działała bez pytania TMDB.
- `on delete cascade`: usunięcie konta usuwa jego filmy.

Row Level Security jest włączone, a trzy zasady pozwalają zalogowanemu użytkownikowi czytać, dodawać i usuwać tylko swoje wiersze. Bez RLS każdy z kluczem aplikacji czytałby całą tabelę.

Klucz anon trafia do aplikacji i każdy może go z niej wyciągnąć. Bezpieczeństwo daje RLS, a nie tajność klucza. Klucza `service_role` nigdy nie wkładamy do aplikacji, bo omija RLS.

Telefon i konto mają mieć te same filmy, także po pracy bez internetu. SwiftData jest źródłem dla ekranu, Supabase jest kopią w chmurze. Synchronizacja robi trzy kroki, w tej kolejności:

1. Filmy usunięte na telefonie usuwa z konta.
2. Filmy z konta dodaje na telefon. Filmy, które zniknęły z konta, bo usunięto je na innym urządzeniu, usuwa z telefonu.
3. Nowe filmy z telefonu wysyła na konto.

Do tego służy pole `isInCloud` z lab 4 i nowy model `RemovedSavedMovie`. Bez znacznika usunięty offline film wróciłby z konta przy następnej synchronizacji.

Pełne diagramy, także przebiegu logowania i synchronizacji, są w `docs/architecture.md` i `docs/database.md`.

## Zadanie 1: baza w Supabase

Schemat bazy jest w projekcie, w `supabase/migrations`. Nie klikamy tabel w panelu, tylko wysyłamy migrację:

```
supabase login
supabase link --project-ref ID_PROJEKTU
supabase db push --include-seed
```

`ID_PROJEKTU` to część adresu panelu po `/project/`. `db push` uruchamia migracje, których projekt jeszcze nie ma. `--include-seed` uruchamia też `supabase/seed.sql`, który tworzy konto testowe `test@test.test` z hasłem `test`.

Proszę przeczytać migrację. Tabelę, RLS i trzy zasady omawiałem na wykładzie. Proszę sprawdzić w panelu:

1. Table Editor: tabela `saved_movies` jest i nie ma przy niej napisu Unrestricted.
2. Authentication, Users: jest konto `test@test.test`.
3. Authentication, Sign In / Providers, Email: „Confirm email” wyłączone. Wtedy nowe konto loguje się od razu. Z włączonym aplikacja ma działać też, ale nowy użytkownik musi najpierw kliknąć link z maila.

Klucze do `Config/Secrets.xcconfig`, z panelu: Project Settings, API.

```
SUPABASE_URL = https:/$()/ID_PROJEKTU.supabase.co
SUPABASE_ANON_KEY = klucz_anon
```

W xcconfig `//` zaczyna komentarz, dlatego w adresie jest `$()`, czyli pusty napis. Po zmianie kluczy proszę zrobić Product, Clean Build Folder (⇧⌘K).

## Zadanie 2: konto

### `Services/Account.swift`

Ten sam pomysł co `MovieLoader`: view model dostaje funkcje i nie wie nic o Supabase. Aplikacja podaje funkcje Supabase, testy zamknięcia.

```
typealias AccountRequest = (_ email: String, _ password: String) async throws
-> String
```

`AccountRequest` loguje albo zakłada konto i zwraca email zalogowanego użytkownika. Pusty napis znaczy, że konto czeka na potwierdzenie maila.

Struktura `AccountService` z czterema polami: `signedInEmail: @MainActor () -> String`, `signIn: AccountRequest`, `signUp: AccountRequest`, `signOut: () async throws -> Void`.

### `Services/Supabase.swift`

`import Supabase` na górze pliku.

Stałe: nazwa tabeli `saved_movies`, kolumny `id` i `saved_at`, klucz `"user_id,id"`, `minimumPasswordLength = 6`, konto testowe `testAccountEmail = "test@test.test"` i `testAccountPassword = "test"`.

`SupabaseError`: jak `TMDBError` z lab 3, pole `message` i `errorDescription`.

Klient to jeden obiekt na całą aplikację, tworzony przy pierwszym zapytaniu:

```
SupabaseClient(supabaseURL: adresProjektu, supabaseKey: kluczAnon)
```

Proszę go trzymać w prywatnej zmiennej globalnej i tworzyć w prywatnej funkcji `requireSupabaseClient() throws -> SupabaseClient`. Gdy w `supabaseURL()` nie ma poprawnego adresu albo `supabaseAnonKey()` jest pusty, funkcja rzuca `SupabaseError` z komunikatem „Missing Supabase keys. Add them to Config/Secrets.xcconfig.”.

| Funkcja | Co robi |
|---|---|
| `supabaseSignedInEmail() -> String` | `client.auth.currentUser?.email`, `""` gdy nikt nie jest zalogowany albo nie ma kluczy |
| `signInToSupabase(email: String, password: String) async throws -> String` | `client.auth.signIn(email:password:)`, zwraca email z sesji |
| `signUpToSupabase(email: String, password: String) async throws -> String` | `client.auth.signUp(email:password:)`. Gdy odpowiedź nie ma sesji, zwraca `""` |
| `signOutOfSupabase() async throws` | `try? await client.auth.signOut()`. Błąd serwera pomijamy, wylogowanie działa też bez internetu |

Stała `supabaseAccountService`: `AccountService` z tymi czterema funkcjami.

Klient trzyma sesję w pęku kluczy. Po ponownym uruchomieniu użytkownik jest dalej zalogowany, a `currentUser` działa bez internetu.

### `ViewModels/AccountViewModel.swift`

Klasa `AccountViewModel`, `@Observable`:

- pola do edycji: `email`, `password`,
- pola `private(set)`: `signedInEmail` (pusty, gdy nikt nie jest zalogowany), `state: LoadState` i `errorMessage` dla logowania, `isWaitingForEmailConfirmation`, `syncState: LoadState` i `syncErrorMessage` dla synchronizacji,
- `init(account: AccountService, syncSavedMovies: @escaping () async throws -> Void, clearSavedMovies: @escaping @MainActor () -> Void)`. Już w `init` czyta `account.signedInEmail()`, żeby zalogowany użytkownik nie zobaczył ekranu logowania nawet na moment.

| **Właściwość albo metoda** | **Co robi** |
|---|---|
| `isSignedIn` | `signedInEmail` nie jest pusty |
| `canSignIn` | email po obcięciu spacji i hasło nie są puste, a logowanie nie trwa |
| `canSignUp` | jak `canSignIn` i hasło ma co najmniej `minimumPasswordLength` znaków |
| `signIn()` | woła `account.signIn` z obciętym emailem. Po sukcesie czyści hasło i synchronizuje. Po błędzie stan `.failed` i komunikat |
| `signUp()` | to samo z `account.signUp`. Pusty wynik ustawia `isWaitingForEmailConfirmation` |
| `syncSavedMovies()` | woła synchronizację i ustawia `syncState`. Gdy nikt nie jest zalogowany, nic nie robi |
| `signOut()` | najpierw synchronizuje ostatnie zmiany, potem `account.signOut()`, czyści `signedInEmail` i woła `clearSavedMovies` |

Logowanie i zakładanie konta to ta sama ścieżka z inną funkcją. Proszę ją wydzielić do prywatnej funkcji, np. `submit(_ request: AccountRequest)`.

Gdy synchronizacja zostanie wywołana w trakcie innej, proszę nie startować drugiej naraz. Wystarczy zapamiętać, że była prośba, i po skończonej synchronizacji zrobić jeszcze jedną. Dwie synchronizacje jednocześnie zmieniałyby listę w tym samym czasie.

### `Views/SignInView.swift`

Pierwszy ekran aplikacji. Dostaje `@Bindable var viewModel: AccountViewModel`, bo pola tekstowe potrzebują bindingu do pól view modelu.

- Nagłówek: ikona filmu, tytuł „My Next Movie” i jedno zdanie, po co jest konto.
- `TextField` na email (bez wielkich liter i autokorekty, klawiatura do maili) i `SecureField` na hasło. Return w haśle loguje.
- Przyciski „Sign In” i „Create Account”, wyłączone, gdy `canSignIn` albo `canSignUp` jest `false`.
- Pod spodem: `ProgressView` w trakcie, komunikat błędu po błędzie, prośba o kliknięcie linku z maila, gdy konto czeka na potwierdzenie, a inaczej informacja o minimalnej długości hasła.

### `Views/AccountView.swift`

Zakładka „Account” z ikoną `person.crop.circle`. `Form` z sekcjami:

- „Signed in as”: email.
- „Saved movies”: stan synchronizacji. W trakcie „Syncing with your account”, po sukcesie „In sync with your account”, po błędzie komunikat i przycisk „Try again”.
- Przycisk „Sign Out”, a pod nim informacja, że zapisane filmy zostają na koncie i znikają z tego urządzenia.

Podglądy obu ekranów potrzebują view modelu bez Supabase. Proszę dodać funkcję `previewAccountViewModel(signedInEmail: String) -> AccountViewModel`, która daje `AccountService` z zamknięciami przyjmującymi dowolny email i hasło.

### `MyNextMovieApp.swift`

`RootView` tworzy `AccountViewModel` z `supabaseAccountService` i dwoma zamknięciami, które wołają funkcje z kontekstem SwiftData: `syncSavedMovies(modelContext, cloud: supabaseSavedMoviesCloud)` z zadania 3 i `clearSavedMovies(modelContext)`.

Nowy widok `AppTabs` dostaje ten view model i loadery z lab 4. Gdy `isSignedIn`, pokazuje zakładki Movies, Saved, Account i Search. Inaczej `SignInView`.
W buildzie Debug ekran logowania ma wpisane konto testowe. Logowanie to wtedy jedno kliknięcie:

```
#if DEBUG
    viewModel.email = testAccountEmail
    viewModel.password = testAccountPassword
#endif
```

Kod w `#if DEBUG` w ogóle nie trafia do wersji Release.

## Testy w `MyNextMovieTests/AccountViewModelTests.swift`

Bez Supabase i bez sieci. Konto udaje prywatna klasa w pliku testów, np. `TestAccount`, z polami: zalogowany email, czy jest offline, czy konto wymaga potwierdzenia maila, ile razy wywołano synchronizację i czyszczenie. Funkcja pomocnicza tworzy z niej `AccountViewModel` z zamknięciami, które czytają i zmieniają te pola.

| Test | Sprawdza |
|---|---|
| `startsSignedOut` | bez sesji `isSignedIn` to `false` |
| `newAccountNeedsLongEnoughPassword` | krótkie hasło pozwala się zalogować, ale nie założyć konta |
| `signInTrimsEmailClearsPasswordAndSyncs` | email bez spacji, hasło puste, jedna synchronizacja |
| `showsErrorWhenSignInFails` | offline: stan `.failed` i komunikat, użytkownik niezalogowany |
| `signUpSignsInWhenNoConfirmationIsNeeded` | nowe konto od razu zalogowane |
| `signUpWaitsForEmailConfirmation` | `isWaitingForEmailConfirmation`, użytkownik niezalogowany |
| `userFromLastTimeIsSignedInAtOnce` | sesja z poprzedniego razu, zalogowany od razu po `init` |
| `doesNotSyncWhenSignedOut` | bez logowania synchronizacja nic nie woła |
| `showsErrorWhenSyncFails` | błąd synchronizacji: `syncState .failed` i komunikat |
| `syncWorksAgainAfterFailure` | po błędzie kolejna synchronizacja kończy się `.loaded` |
| `signOutSyncsThenClearsDevice` | wylogowanie: najpierw synchronizacja, potem czyszczenie, użytkownik niezalogowany |

Proszę sprawdzić w aplikacji:

1. Pierwszy ekran to logowanie, z wpisanym kontem testowym.
2. Po zalogowaniu są cztery zakładki, a Account pokazuje `test@test.test`.
3. Po zamknięciu i ponownym uruchomieniu użytkownik jest dalej zalogowany.
4. „Sign Out” wraca do ekranu logowania.
5. Złe hasło pokazuje komunikat błędu.

## Zadanie 3: synchronizacja

`Storage/StoredModels.swift`

- `SavedMovie` ma już `isInCloud`. `init` dostaje trzeci parametr: `init(movie: Movie, savedAt: Date, isInCloud: Bool)`.
- Nowy model `RemovedSavedMovie` z jednym polem `movieId: Int` (unikalne). Znaczy: film usunięto na telefonie, ale jeszcze nie z konta. Film jest albo w `SavedMovie`, albo w `RemovedSavedMovie`, nigdy w obu. Proszę go dopisać do `storedModelTypes`.

### `Storage/Storage.swift`

- `toggleSavedMovie` zapisuje nowy film z `isInCloud: false`. Gdy usuwa film, który ma `isInCloud`, zostawia `RemovedSavedMovie` z jego id. Film, którego konto jeszcze nie miało, znika bez znacznika.
- Nowa funkcja `clearSavedMovies(_ context: ModelContext)` dla wylogowania: usuwa wszystkie `SavedMovie` i wszystkie `RemovedSavedMovie` z telefonu. Konto zachowuje swoje filmy. Proszę usuwać po jednym w pętli, jak w lab 4.

### `Services/Account.swift`

Struktura `SavedMoviesCloud` z trzema polami: `fetchMovies: () async throws -> [CloudSavedMovie]`, `uploadMovies: (_ cloudMovies: [CloudSavedMovie]) async throws -> Void`, `removeMovie: (_ movieId: Int) async throws -> Void`. To samo co `AccountService`, tylko dla tabeli.

### `Services/Supabase.swift`

`CloudSavedMovie`: jeden wiersz `saved_movies`, struktura `Codable` i `Equatable`, oznaczona `nonisolated`. Pola `Movie` (`id`, `title`, `overview`, `releaseDate`, `posterPath`, `voteAverage`, `genreIds`) i `savedAt: Date`. Bez `user_id`, bo bazę wypełnia go sama. Nazwy kolumn są w snake_case, więc `CodingKeys` z `case releaseDate = "release_date"` i tak dalej. Pole bez napisu ma taką samą nazwę jak kolumna.

| **Funkcja** | **Co robi** |
|---|---|
| `fetchCloudSavedMovies() async throws -> [CloudSavedMovie]` | wiersze tabeli, najnowsze pierwsze. Bez warunku na użytkownika, RLS i tak zwraca tylko jego wiersze |
| `uploadCloudSavedMovies(_ cloudMovies: [CloudSavedMovie]) async throws` | `upsert` z `onConflict: "user_id,id"`, `returning: .minimal`, `ignoreDuplicates: true`. Pusta lista nie wysyła nic |
| `removeCloudSavedMovie(movieId: Int) async throws` | `delete(returning: .minimal)` z `.eq("id", value: movieId)` |

Odczyt to łańcuch: tabela, `select`, sortowanie, wykonanie. `value` dekoduje JSON do typu podanego po lewej stronie:

```
let cloudMovies: [CloudSavedMovie] = try await client
    .from("saved_movies")
    .select()
    .order("saved_at", ascending: false)
    .execute()
    .value
```

Stała `supabaseSavedMoviesCloud`: `SavedMoviesCloud` z tymi trzema funkcjami.

`upsert` dodaje wiersze, a przy tym samym kluczu nie tworzy duplikatu. Z `ignoreDuplicates` nie zmienia też istniejących wierszy.

`Storage/SavedMoviesSync.swift`

| Funkcja | Co robi |
|---|---|
| `syncSavedMovies(_ context: ModelContext, cloud: SavedMoviesCloud) async throws` | trzy kroki z części „Jak działa Supabase” |
| `cloudMovieFromSaved(_ savedMovie: SavedMovie) -> CloudSavedMovie` | wiersz do wysłania |
| `movieFromCloud(_ cloudMovie: CloudSavedMovie) -> Movie` | film z wiersza |

Zasady:

- Krok 1: dla każdego `RemovedSavedMovie` `cloud.removeMovie`, potem usunięcie znacznika.
- Krok 2: `cloud.fetchMovies()`. Film z `isInCloud`, którego konto już nie ma, znika z telefonu. Film z konta, którego telefon nie ma, dochodzi z `isInCloud: true` i datą z konta. Film, który telefon już ma, dostaje `isInCloud = true`. Film ze znacznikiem usunięcia nie wraca.
- Krok 3: wszystkie `SavedMovie` z `isInCloud == false` idą jednym `cloud.uploadMovies`, potem dostają `isInCloud = true`.
- Gdy zapytanie się nie uda, funkcja rzuca błąd. Niewysłane zmiany zostają oznaczone i pójdą przy następnej synchronizacji.
- W trakcie `await` użytkownik może zapisać albo usunąć film. Dlatego po każdym `await` proszę czytać bazę od nowa, a przez `await` przenosić tylko id filmów. Film usunięty w trakcie wysyłania dostaje po wysłaniu znacznik `RemovedSavedMovie`, żeby zniknął też z konta.

Każdy krok proszę wydzielić do prywatnej funkcji.

### `MyNextMovieApp.swift`: kiedy synchronizować

`AppTabs` woła `account.syncSavedMovies()`:

- przy starcie: `.task`,
- po powrocie do aplikacji: `.onChange(of: scenePhase)`, gdy faza to `.active`,
- po zapisaniu albo usunięciu filmu: gdy rośnie liczba niewysłanych zmian. To suma dwóch `@Query`: `SavedMovie` z `isInCloud == false` i wszystkich `RemovedSavedMovie`. Spadek liczby to skutek synchronizacji i nie wymaga następnej.

## Testy

Synchronizację testujemy bez Supabase. Tabelę jednego konta udaje tablica w prywatnej klasie, np. `TestCloud` z polami `movies: [CloudSavedMovie]` i `isOffline: Bool`. Klasa jest potrzebna, żeby zamknięcia w `SavedMoviesCloud` i test widziały te same wiersze. Do tego baza SwiftData w pamięci z `makeTestContainer` z lab 4.

`MyNextMovieTests/SavedMoviesSyncTests.swift`, struktura `SavedMoviesSyncTests`:

| **Test** | **Sprawdza** |
|---|---|
| `movieSavedOnDeviceGoesToAccount` | film zapisany na telefonie trafia na konto i ma `isInCloud` |
| `movieIsSentOnlyOnce` | druga synchronizacja nie wysyła go ponownie |
| `movieFromAccountComesToDevice` | film z konta pojawia się na telefonie |
| `deviceAndAccountListsAreMerged` | po synchronizacji telefon i konto mają sumę obu list |
| `movieRemovedOnDeviceIsRemovedFromAccount` | usunięcie na telefonie usuwa z konta i znacznik znika |
| `movieRemovedOnAnotherDeviceIsRemovedHere` | film, którego konto już nie ma, znika z telefonu |
| `changesMadeOfflineAreSentWithTheNextSync` | offline: usunięcie jednego filmu i zapis drugiego, synchronizacja rzuca błąd i konto się nie zmienia. Następna synchronizacja wysyła obie zmiany |
| `movieRemovedAndSavedAgainOfflineStays` | film usunięty i znów zapisany przed synchronizacją zostaje na koncie, z nową datą zapisu |

Struktura `CloudSavedMovieTests` w tym samym pliku:

| **Test** | **Sprawdza** |
|---|---|
| `encodesColumnNamesInSnakeCase` | JSON z `CloudSavedMovie` ma klucze `release_date`, `poster_path`, `vote_average`, `genre_ids`, `saved_at` |
| `cloudCopyKeepsEveryMovieField` | `movieFromCloud(cloudMovieFromSaved(...))` daje ten sam film |

Do `StorageTests.swift`, struktura `SavedMovieTests`:

| **Test** | **Sprawdza** |
|---|---|
| `removingMovieTheAccountHasLeavesAMark` | usunięcie filmu z `isInCloud` zostawia `RemovedSavedMovie` |
| `removingMovieOnlyOnDeviceLeavesNoMark` | film, którego konto nie miało, znika bez znacznika |
| `clearRemovesMoviesAndMarks` | `clearSavedMovies` usuwa filmy i znaczniki |

Proszę sprawdzić w aplikacji:

1. Zapisany film pojawia się w Table Editor w tabeli `saved_movies`.
2. Po usunięciu aplikacji z symulatora i ponownej instalacji zapisane filmy wracają po zalogowaniu.
3. Drugi symulator z tym samym kontem ma te same filmy. Zmiana na jednym pojawia się na drugim po powrocie do aplikacji.
4. Bez internetu (wyłączone Wi-Fi w Macu) zapis i usunięcie działają, a zakładka Account pokazuje błąd synchronizacji. Po włączeniu internetu i powrocie do aplikacji zmiany trafiają na konto.
5. Po wylogowaniu lista znika z telefonu, a po ponownym zalogowaniu wraca.

## Dokumentacja

- Supabase, biblioteka dla Swifta: https://supabase.com/docs/reference/swift
- Supabase, logowanie emailem: https://supabase.com/docs/guides/auth/passwords
- Supabase, Row Level Security: https://supabase.com/docs/guides/database/postgres/row-level-security
- Supabase CLI: https://supabase.com/docs/guides/local-development/cli/getting-started
- Supabase, migracje: https://supabase.com/docs/guides/deployment/database-migrations
- `@Bindable`: https://developer.apple.com/documentation/swiftui/bindable
- `scenePhase`: https://developer.apple.com/documentation/swiftui/scenephase
- `SecureField`: https://developer.apple.com/documentation/swiftui/securefield